Energy Efficiency in Railway Scheduling

Paul Tennigkeit and Georgi Abshilava

University of Potsdam, Germany
{tennigkeit, abshilava}@uni-potsdam.de

1 Introduction

2 Encoding Energy Efficiency in ASP

2.1 What is Flatland?

Flatland is a framework for simulating railway networks. A Flatland environment consists of a two dimensional plane made up of different types of cells, some of which containing different types of interconnected tracks, and agents moving around on them from a starting point to an end point. Agents have an earliest departure and arrival time, and a pre-defined speed, represented by an inverse value. To limit the time of the simulation, a global time frame is set.

2.2 Baseline Encoding

The baseline ASP encoding (train_encoding.lp) handles all essential train and environment behavior, including defining legal transitions and movements, possible directions, and actions.

Directions There are four possible directions agents can go and point towards: North, south, east and west. Each is defined as a fact:

```
dir(n). dir(s). dir(e). dir(w).
```

Movements Each possible movement is defined as a fact:

```
moving(move_forward).
moving(move_left).
moving(move_right).
```

`delta()` defines what it means to "move" in each direction:

```
% The origin is located in the top left in Flatland, and
% the coordinates increment downwards

delta(A,s,1,0)  :- moving(A).
delta(A,n,-1,0) :- moving(A).
delta(A,e,0,1)  :- moving(A).
delta(A,w,0,-1) :- moving(A).
```

Transitions In order to represent different types of tracks, `transitions()` predicates are defined for each:

```
% ID  : track ID
% Dir1: pointing direction of agent
% Act : associated action with movement towards exit
% Dir2: exit direction
transition(ID, Dir1, Act, Dir2).
```

Due to the way this predicate is built, each track type is defined multiple times depending on its number of exit directions (e.g. straight tracks have two possible exits, north and south, and are, therefore, defined twice for each possible exit).

Actions The `action()` predicate is used as an interface between the Flatland visualization engine and the Clingo logic:

```
% train(ID): A train with an ID
% A        : An action
% T        : The time step action A is taken
action(train(ID),A,T) :- chosen(ID,T,A,_,_,_,_,_).
action(train(ID),move_forward,Dep) :- start(ID,_,Dep,_),
                                      Dep > 0.

action(train(ID),wait,T) :- start(ID,_,Dep,_),
                            Dep > 0,
                            T = 0..Dep-1.
```

Collision Constraints Like in real life, collisions between trains are to be avoided. To achieve this, the following constraints are employed:

```
% Trains can not occupy the same cell at the same time
```

```
:- stay(ID1,X,Y,T),
   stay(ID2,X,Y,T),
   ID1 < ID2.

% Trains can not swap their position
:- stay(ID1,X1,Y1,T),
   stay(ID2,X2,Y2,T),
   stay(ID1,X2,Y2,T+1),
   stay(ID2,X1,Y1,T+1),
   ID1 < ID2.
```

Reachability Constraints To simulate movement across the grid over time, a train's initial departure parameters are first converted into a `position()` state:

```
position(ID,(X,Y),Dep+1,Dir) :- start(ID,(X,Y),Dep,Dir).
```

From any current position, valid future states are derived using the `reachable()` predicate by checking cell track transitions, movement coordinate deltas, and the train's speed:

```
reachable(ID,T,A,X2,Y2,T2,Dir2) :- A != wait,
                                   position(ID,(X,Y),T,Dir1),
                                   cell((X,Y),Cell),
                                   transition(Cell,Dir1,A,Dir2),
                                   delta(A,Dir2,DX,DY),
                                   X2 = X + DX,
                                   Y2 = Y + DY,
                                   speed(ID,S),
                                   T2 = T + S,
                                   time(T2).

reachable(ID,T,wait,X,Y,T+1,Dir) :- position(ID,(X,Y),T,Dir),
                                    time(T+1).
```

At each discrete time step, a choice rule ensures exactly one valid transition is taken until the destination is reached:

```
1 { position(ID,(X2,Y2),T2,Dir2)
    : reachable(ID,T,A,X2,Y2,T2,Dir2) } 1
    :- position(ID,(_,_),T,_), not reached(ID,T).
```

In order to properly enforce the collision constraints, the time interval a train spends inside a specific cell must be tracked:

```
chosen(ID,T,A,X,Y,X2,Y2,T2) :- position(ID,(X,Y),T,_),
                               position(ID,(X2,Y2),T2,_),
                               reachable(ID,T,A,X2,Y2,T2,_).
```

Because trains can have an inverse speed greater than 1 (meaning they traverse a cell over multiple time steps), the `stay()` predicate marks every time step a cell is occupied between entering and leaving:

```
% for inverse speed !=1: define interval to stay in a cell
stay(ID,X,Y,T1) :- chosen(ID,T,_,X,Y,_,_,T2),
                   time(T1), T <= T1, T1 < T2.
```

Target Arrival A train successfully completes its journey when its current coordinates match its designated target destination:

```
reached(ID,T) :- position(ID,(X,Y),T,_),
                 end(ID,(X,Y),_).
```

To ensure valid and bounded schedules, a look-ahead integrity constraint terminates search branches where a train can physically no longer reach its end coordinates within its individual allowable arrival deadline:

```
% stop searching if train is not in time
% to reach its target anymore
:- position(ID,(X,Y),T,_),
   end(ID,(EX,EY),Arr),
   speed(ID,InvSpd),
   T + (|X-EX| + |Y-EY|)*InvSpd > Arr.
```

By evaluating the remaining Manhattan distance multiplied by the train's inverse speed directly against its scheduled deadline, this single rule simultaneously prevents late arrivals and prunes unviable paths early in the search space.

Schedule Optimization Once valid routing pathways are established, the solver optimizes the schedule by minimizing overall transit delays. To ensure that trains are evaluated solely on their actual transit

time rather than time spent idling at their destination after completion, the encoding isolates the precise time step T at which a train first reaches its end coordinates:

```
% the arrival times should be minimized
reached_earlier(ID,T) :- reached(ID,T),
                         reached(ID,T0),
                         T0 < T.
arrival(ID,T) :- reached(ID,T),
                 not reached_earlier(ID,T).
```

The `#minimize` statement aggregates the arrival times of all agents and assigns this objective a priority level of `@2`. This high priority ensures that in extended multi-objective encodings minimizing passenger schedule delay always takes precedence over secondary optimization metrics.

```
% minimize summed arrival times
#minimize { T@2, ID : arrival(ID,T) }.
```

2.3 Energy Encoding Mechanics

The energy efficiency encoding (train_encoding_energy.lp) adds new rules to represent the concept of energy and how different actions of the agents affect their usage and consumption of energy.

Cost Factors In real life, trains use a variety of motors that rely on different sources of energy, such as electric, hybrid, diesel or hydrogen power, which fundamentally affects how efficiently energy is consumed. The `pow_cost_fac()` predicate represents this baseline efficiency in the encoding by assigning an integer multiplier to each train based on its power type.

To then determine the true energy footprint of an agent, its mass must also be accounted for. This is handled by the `cost_fac()` rule, by multiplying a trains assigned cost factor, the `PCF`, by its defined weight `W`, resulting in a final, individualized cost factor `CF` for each train in the simulation:

```
% energy cost factor of trains based on power type + weight
pow_cost_fac(ID,1) :- pow_type(ID, electro).
```

```
pow_cost_fac(ID,2) :- pow_type(ID, hybrid).
pow_cost_fac(ID,3) :- pow_type(ID, diesel).
pow_cost_fac(ID,4) :- pow_type(ID, hydrogen).

cost_fac(ID,CF) :- pow_cost_fac(ID,PCF),
                   weight(ID,W),
                   CF = PCF * W.
```

The trains' masses and energy types are all set in the environment definition:

```
train(0).
start(0,(7,6),24,w).
end(0,(13,17),71).
speed(0,1).
pow_type(0, electro).
weight(0,2).

train(1).
start(1,(12,17),21,e).
end(1,(8,6),65).
speed(1,1).
pow_type(1, diesel).
weight(1,4).

train(2).
start(2,(12,17),3,w).
end(2,(8,6),94).
speed(2,4).
pow_type(2, hybrid).
weight(2,3).

train(3).
start(3,(9,6),18,w).
end(3,(13,17),65).
speed(3,1).
pow_type(3, hydrogen).
weight(3,3).
```

Energy Costs of Actions Depending how fast the train moves or accelerates, it consumes different amounts of energy. These are reflected by the different `energy()` predicates:

```
% This rule calculates the energy cost of waiting,
% which is CF * 24 / S. The penalty is only applied
% once, when the train first hits the brakes.
% This behavior is reflected by the rule
% not prev_wait(ID,T).
energy(ID,T+1,E) :- action(train(ID),wait,T),
                    speed(ID,S),
                    not prev_wait(ID,T),
                    cost_fac(ID,CF),
                    E = CF * 24 / S.

% This rule calculates the energy cost of moving
% straight. This action's cost is only the base CF.
energy(ID,T2,E) :- action(train(ID),A,T),
                   A != wait,
                   position(ID,_,T,Dir),
                   speed(ID,S),
                   T2 = T + S,
                   position(ID,_,T2,Dir),
                   cost_fac(ID,CF),
                   E = CF .

% This rule calculates the energy cost of turning.
energy(ID,T2,E) :- position(ID,_,T,Dir1),
                   speed(ID,S),
                   T2 = T + S,
                   position(ID,_,T2,Dir2),
                   Dir1 != Dir2,
                   cost_fac(ID,CF),
                   E = CF * 12 / S.

% The prev_wait() rule is required in order to make sure,
% the agent is not constantly penalized for waiting in
% every time step.
```

```
prev_wait(ID,T) :- action(train(ID),_,T),
                   time(T-1),
                   action(train(ID),wait,T-1).
```

Optimizing the Energy Encoding The energy encoding adds the rule `#minimize { E@1,ID,T : energy(ID,T,E) }.` to also optimize fuel efficiency. As it is assigned the weight `@1`, the solver optimizes for arrival time first, which is assigned `@2`. Fuel efficiency is then optimized within the bounds of the best possible arrival times.

2.4 Acceleration Encoding Mechanics

This is an extension to the energy encoding, that also models the trains' acceleration.

Maximum and Initial Speeds The environment created for the experiments also defined the trains' speeds in form of `speed()` predicates, which is actually interpreted as an inverse speed (e.g. a `speed` of 4 actually means, that a train would be 25% slower). To model acceleration, `speed()` is reinterpreted as `max_speed()`.

```
% interpret given train speeds as maximum speeds
max_speed(ID,S) :- speed(ID,S).
```

As a train starts moving, it initially has a speed of 0.25.

```
% trains start with a speed of 1/4
temp_speed(ID, Dep+1, 4) :- start(ID,(_,_),Dep,_).
```

Dynamic Speed Change The `temp_speed()` choice rules allow gradual speed changes while strictly respecting a train's physical limits. During each position change, a train can alter its inverse speed by a maximum of one step at a time.

```
1{
  temp_speed(ID,T2,S-1);
  temp_speed(ID,T2,S);
  temp_speed(ID,T2,S+1)
}1
:- chosen(ID,T,A,_,_,_,_,T2),
```

```
   A != wait,
   temp_speed(ID,T,S),
   max_speed(ID,MS),
   S-1 >= MS,
   S+1 <= 4.
```

To account for the upper and lower bounds of a train's speed capabilities, three additional choice rules handle edge cases: when a train is at its maximum speed, when it is at its minimum speed, or when both conditions apply simultaneously. In the later case, trains are forced to maintain their current speed.

```
% Lower bound
1{
  temp_speed(ID,T2,S-1);
  temp_speed(ID,T2,S)
}1
:- chosen(ID,T,A,_,_,_,_,T2),
   A != wait,
   temp_speed(ID,T,S),
   max_speed(ID,MS),
   S-1 >= MS,
   S+1 > 4.

% Upper bound
1{
  temp_speed(ID,T2,S);
  temp_speed(ID,T2,S+1)
}1
:- chosen(ID,T,A,_,_,_,_,T2),
   A != wait,
   temp_speed(ID,T,S),
   max_speed(ID,MS),
   S-1 < MS,
   S+1 <= 4.

% In between
temp_speed(ID,T2,S)
:- chosen(ID,T,A,_,_,_,_,T2),
```

```
   A != wait,
   temp_speed(ID,T,S),
   max_speed(ID,MS),
   S-1 < MS,
   S+1 > 4.
```

Post-Wait Acceleration When a train decides to wait, it loses its momentum. The encoding dictates that after taking a `wait` action, a train must resume movement at the lowest possible speed, which corresponds to an inverse speed of 4.

```
% after waiting, you have to start with speed 1/4
temp_speed(ID,T2,4)  :- chosen(ID,T,wait,_,_,_,_,T2).

% to be able to wait, you need to have speed 1/4
reachable(ID,T,wait,X,Y,T+1,Dir)
:- position(ID,(X,Y),T,Dir),
   time(T+1),
   temp_speed(ID,T,4).
```

Turning and Arrival Physics Realistic physics dictate that trains must decelerate before taking sharp turns or arriving at their destination. The model enforces that a train cannot take a turn at high speeds; it must slow down to an inverse speed of at least 3. Similarly, it must fully decelerate to an inverse speed of 4 upon reaching its final target.

```
% you have to take a turn with inverse speed >= 3
:- position(ID,_,T,Dir1),
   temp_speed(ID,T,S),
   T2 = T + S,
   position(ID,_,T2,Dir2),
   Dir1 != Dir2,
   S < 3.

% you can only arrive at your goal if your inverse speed was 4 before
:- reached(ID,T2),
   temp_speed(ID,T1,S),
   T2 = T1 + S,
   S < 4.
```

Updated Energy Costs The introduction of acceleration changes how energy consumption is calculated compared to the energy encoding, that does not take acceleration into account. The cost equations now heavily depend on the current inverse speed `S` and apply scaling multipliers to moving, waiting, and turning.

```
% Energy  cost of waiting
energy(ID,T+1,E) :- action(train(ID),wait,T),
                    not prev_wait(ID,T),
                    cost_fac(ID,CF),
                    E = -1 * 100 * 4/10 * CF * 1/4 * 1/4.

% Energy  cost of moving
energy(ID,T2,E) :- action(train(ID),A,T),
                   A != wait,
                   position(ID,_,T,Dir),
                   temp_speed(ID,T,S),
                   T2 = T + S,
                   position(ID,_,T2,Dir),
                   cost_fac(ID,CF),
                   E = 100 * CF * 1/S * 1/S.

% Energy  cost of turning
energy(ID,T2,E) :- position(ID,_,T,Dir1),
                   temp_speed(ID,T,S),
                   T2 = T + S,
                   position(ID,_,T2,Dir2),
                   Dir1 != Dir2,
                   cost_fac(ID,CF),
                   E = 100 * CF * 1/S * 1/S  * 11/10.
```

Solver Heuristics To guide the solver's search process through the expanded state space, the encoding introduces specific `#heuristic` directives to prioritize specific branches by penalizing waiting, scheduling trains one by one based on departure times, and instructing the solver to prefer paths where trains gain speed quickly.

```
% prefer to not wait
wait_position(ID,X2,Y2,T2,Dir2)
```

```
:- reachable(ID,T,wait,X2,Y2,T2,Dir2).

#heuristic position(ID,(X2,Y2),T2,Dir2)
           : wait_position(ID,X2,Y2,T2,Dir2). [-50,1]

% schedule trains one by one
#heuristic position(ID,(X,Y),T,Dir)
           : start(ID,_,Dep,_). [-Dep,1]

% prefer getting faster
#heuristic temp_speed(ID,T,S)
           : position(ID,_,T,_). [-S,1]
```

2.5 Results

For all trial runs a single environment was used. It consists of a 25 $\times$ 25 network with two stations and the following four agents:

```
train(0).
start(0,(7,6),24,w).
end(0,(13,17),71).
speed(0,1).
pow_type(0, electro).
weight(0,2).

train(1).
start(1,(12,17),21,e).
end(1,(8,6),65).
speed(1,1).
pow_type(1, diesel).
weight(1,4).

train(2).
start(2,(12,17),3,w).
end(2,(8,6),94).
speed(2,4).
pow_type(2, hybrid).
weight(2,3).
```

```
train(3).
start(3,(9,6),18,w).
end(3,(13,17),65).
speed(3,1).
pow_type(3, hydrogen).
weight(3,3).
```

Fig. 1. Flatland environment used for all experiments

The baseline encoding was only used to establish rules for bringing trains from a starting point to an end point with a proper behavior for a given environment. That goal was reached successfully: The baseline encoding successfully generated paths for all four agents from their starting point to their end points (see tables **??**, **??**, **??** and **??**).

Looking at the energy values in the path finding logs reveals, that the local accounting of energy consumption operates in accordance with the theoretical modeling.

Elevated Energy Consumption when Turning In physical rail operations, negotiating curves entails additional lateral forces, wheel flange friction, and curve resistance, resulting in a substantially higher instantaneous energy demand compared to straight cruising.

This dynamic is accurately reflected in the experimental path logs (see tables **??**, **??**, **??**, and **??**):

- **Straight Motion:** While maintaining a constant heading, agents only consume their base cost factor ($E = CF$). Agent 0 (electric, $CF = 2$) expends 2 units per step during straight traversal, whereas the heavier diesel and hydrogen units (Agents 1 and 3, $CF = 12$) consume 12 units per step.
- **Turning Penalties:** Whenever an agent alters its orientation between successive positions ($Dir1 \neq Dir2$), the energy calculation rule $E = CF \cdot 12/S$ is triggered. This leads to pronounced consumption peaks:
  - For Agent 0 ($S = 1$), energy consumption spikes twelvefold from 2 to 24 units (e.g., timesteps 27, 28, and 32).
  - For Agents 1 and 3 ($S = 1$), turning actions incur a severe penalty of 144 units instead of 12 (e.g., Agent 1 at timesteps 28 and 33; Agent 3 at timesteps 23, 26, and 31).
  - For Agent 2, whose inverse speed is $S = 4$, the penalty scales inversely with speed to $E = 6 \cdot 12/4 = 18$ units (e.g., timestep 33), representing a threefold increase over its baseline of 6 units.

These observations confirm that the ASP encoding successfully differentiates between vehicle types, propulsion efficiencies, and kinematic states on an individual step level.

Mass and Propulsion Scaling The linear coupling of drive efficiency and vehicle weight ($CF = PCF \cdot W$) correctly scales the baseline energy footprint. Agent 0, representing the lightest configuration ($W = 2$, electric), operates at a minimal baseline of 2 units per step. In contrast, Agent 1, having the highest mass ($W = 4$, diesel),

requires 12 units per step—a sixfold increase that accurately reflects the physical impact of vehicle mass and motor efficiency on running resistance.

Braking Penalties As mentioned in chapter 2.3, the energy encoding also specifies penalties when a train first enters the `wait` state, which models a train first hitting the brakes. The path finding logs highlight two distinct behaviors regarding this rule:

- **Avoidance During Transit:** Because braking incurs a massive energy penalty ($E = CF \cdot 24/S$), the solver actively avoids scheduling `wait` actions once a train is in motion. By successfully leaning on the `#minimize` directive for energy, the solver guarantees that trains maintain their momentum, completely eliminating unnecessary stops during their active routes.
- **Initialization Boundary Condition:** An initialization artifact is visible at timestep 1 for all agents (see tables **??**–**??**). The encoding determines a new braking action by checking if an agent was already waiting in the previous timestep (`not prev_wait(ID,T)`). At the absolute start of the simulation ($T = 0$), the absence of a preceding timestep causes this condition to evaluate as true. Consequently, the model mathematically assesses the initial static state at the station as a braking maneuver, applying the strict formula. This correctly yields the expected theoretical penalty peaks: 48 units for Agent 0, 288 units for Agents 1 and 3, and 36 units for Agent 2.

Ultimately, while the boundary condition at $T = 0$ generates an initial offset, the encoding successfully forces the solver to prioritize continuous, momentum-preserving movement throughout the actual journey.

Computational Complexity While the encoding successfully navigated the multi-objective optimization, the introduction of energy mechanics heavily impacted solver efficiency. Establishing the optimal path for this 25×25 environment required 3021.327 seconds of computation time, with roughly 2999.51 seconds dedicated purely to solving. This exponential increase in search space complexity is driven by the `energy()` rules, which force the solver to evaluate highly granular kinematic constraints across extended time horizons.

3 Conclusion

In this project, we extended the railway scheduling problem within the Flatland framework by modeling energy consumption using Answer Set Programming (ASP). Our results demonstrate that the ASP encoding accurately captures physical realities at the local level: energy costs dynamically scale based on vehicle mass, power type, and specific kinematic states such as turning or braking. Furthermore, the application of lexicographical optimization effectively dictates solver behavior. By prioritizing arrival time over energy efficiency, the solver strictly avoids costly braking maneuvers to maintain momentum, ultimately producing paths identical to the baseline while optimizing transit continuity.

However, our current ASP encoding exhibits limitations regarding computational scalability. The introduction of complex energy penalties expands the search space significantly, particularly because the solver must evaluate numerous kinematic combinations across extended time horizons.

We consider several extensions and modifications for future work based on our findings:

- **Alternative Optimization Strategies:** Adjusting or combining the `#minimize` directives—such as applying a mathematical scaling factor to unify time and energy costs—could allow the solver to actively evaluate and choose longer, more energy-efficient detours rather than strictly defaulting to the fastest geometric route.
- **Algorithmic Improvements:** To manage the exponential growth of the search space on larger grids, future research could employ incremental solving techniques. Progressively extending the planning horizon could help approximate optimal energy solutions more efficiently without grounding the entire time horizon at once.
- **Acceleration and Kinematics:** While this model utilizes constant inverse speeds, future iterations could incorporate dynamic acceleration mechanics, adding a layer of realism to how trains gather momentum after stops or decelerate before sharp turns.

Overall, we provide a robust proof of concept that ASP can effectively model and optimize deep physical constraints in railway logis-

tics, laying the groundwork for future research into multi-objective routing.

A Tables for Baseline Encoding

Table 1. Path taken by agent 0 in the baseline encoding

agent	timestep	position	direction	status	given_command
0	0	None	w	waiting	wait
0	1	None	w	waiting	wait
0	2	None	w	waiting	wait
0	3	None	w	waiting	wait
0	4	None	w	waiting	wait
0	5	None	w	waiting	wait
0	6	None	w	waiting	wait
0	7	None	w	waiting	wait
0	8	None	w	waiting	wait
0	9	None	w	waiting	wait
0	10	None	w	waiting	wait
0	11	None	w	waiting	wait
0	12	None	w	waiting	wait
0	13	None	w	waiting	wait
0	14	None	w	waiting	wait
0	15	None	w	waiting	wait
0	16	None	w	waiting	wait
0	17	None	w	waiting	wait
0	18	None	w	waiting	wait
0	19	None	w	waiting	wait
0	20	None	w	waiting	wait
0	21	None	w	waiting	wait
0	22	None	w	waiting	wait
0	23	None	w	waiting	wait
0	24	None	w	ready to depart	move_forward
0	25	(7, 6)	w	moving	move_forward
0	26	(7, 5)	w	moving	move_forward
0	27	(8, 5)	s	moving	move_forward
0	28	(8, 4)	w	moving	move_forward
0	29	(8, 3)	w	moving	move_forward
0	30	(8, 2)	w	moving	move_forward
0	31	(8, 1)	w	moving	move_forward
0	32	(9, 1)	s	moving	move_forward
0	33	(10, 1)	s	moving	move_forward
0	34	(11, 1)	s	moving	move_forward
0	35	(12, 1)	s	moving	move_forward
0	36	(13, 1)	s	moving	move_forward
0	37	(13, 2)	e	moving	move_forward
0	38	(13, 3)	e	moving	move_forward
0	39	(13, 4)	e	moving	move_forward
0	40	(13, 5)	e	moving	move_forward
0	41	(13, 6)	e	moving	move_forward
0	42	(13, 7)	e	moving	move_forward
0	43	(13, 8)	e	moving	move_forward
0	44	(13, 9)	e	moving	move_forward
0	45	(13, 10)	e	moving	move_forward
0	46	(13, 11)	e	moving	move_forward
0	47	(13, 12)	e	moving	move_forward
0	48	(12, 12)	n	moving	move_right
0	50	(12, 14)	e	moving	move_right
0	51	(13, 14)	s	moving	move_forward
0	52	(13, 15)	e	moving	move_forward
0	53	(13, 16)	e	moving	move_forward

Table 2. Path taken by agent 1 in the baseline encoding

agent	timestep	position	direction	status	given_command
1	0	None	e	waiting	wait
1	1	None	e	waiting	wait
1	2	None	e	waiting	wait
1	3	None	e	waiting	wait
1	4	None	e	waiting	wait
1	5	None	e	waiting	wait
1	6	None	e	waiting	wait
1	7	None	e	waiting	wait
1	8	None	e	waiting	wait
1	9	None	e	waiting	wait
1	10	None	e	waiting	wait
1	11	None	e	waiting	wait
1	12	None	e	waiting	wait
1	13	None	e	waiting	wait
1	14	None	e	waiting	wait
1	15	None	e	waiting	wait
1	16	None	e	waiting	wait
1	17	None	e	waiting	wait
1	18	None	e	waiting	wait
1	19	None	e	waiting	wait
1	20	None	e	waiting	wait
1	21	None	e	ready to depart	move_forward
1	22	(12, 17)	e	moving	move_forward
1	23	(12, 18)	e	moving	move_forward
1	24	(12, 19)	e	moving	move_forward
1	25	(12, 20)	e	moving	move_forward
1	26	(12, 21)	e	moving	move_forward
1	27	(12, 22)	e	moving	move_forward
1	28	(11, 22)	n	moving	move_forward
1	29	(10, 22)	n	moving	move_forward
1	30	(9, 22)	n	moving	move_forward
1	31	(8, 22)	n	moving	move_forward
1	32	(7, 22)	n	moving	move_forward
1	33	(7, 21)	w	moving	move_forward
1	34	(7, 20)	w	moving	move_forward
1	35	(7, 19)	w	moving	move_forward
1	36	(7, 18)	w	moving	move_forward
1	37	(7, 17)	w	moving	move_forward
1	38	(7, 16)	w	moving	move_forward
1	39	(7, 15)	w	moving	move_forward
1	40	(7, 14)	w	moving	move_forward
1	41	(7, 13)	w	moving	move_forward
1	42	(7, 12)	w	moving	move_left
1	43	(8, 12)	s	moving	move_forward
1	44	(8, 11)	w	moving	move_forward
1	45	(8, 10)	w	moving	move_forward
1	46	(8, 9)	w	moving	move_forward
1	47	(8, 8)	w	moving	move_forward
1	48	(8, 7)	w	moving	move_forward

Table 3. Path taken by agent 2 in the baseline encoding

agent	timestep	position	direction	status	given_command
2	0	None	w	waiting	wait
2	1	None	w	waiting	wait
2	2	None	w	waiting	wait
2	3	None	w	ready to depart	move_forward
2	4	(12, 17)	w	moving	move_forward
2	8	(12, 16)	w	moving	move_forward
2	12	(12, 15)	w	moving	move_forward
2	16	(12, 14)	w	moving	move_forward
2	20	(12, 13)	w	moving	move_forward
2	24	(12, 12)	w	moving	move_forward
2	28	(12, 11)	w	moving	move_forward
2	32	(11, 11)	n	moving	move_forward
2	36	(10, 11)	n	moving	wait
2	37	(10, 11)	n	stopped	move_forward
2	41	(9, 11)	n	moving	move_forward
2	45	(8, 11)	n	moving	move_left
2	49	(8, 10)	w	moving	move_forward
2	53	(8, 9)	w	moving	move_forward
2	54	(8, 9)	w	moving	move_forward
2	55	(8, 9)	w	moving	move_forward

Table 4. Path taken by agent 3 in the baseline encoding

agent	timestep	position	direction	status	given_command
3	0	None	w	waiting	wait
3	1	None	w	waiting	wait
3	2	None	w	waiting	wait
3	3	None	w	waiting	wait
3	4	None	w	waiting	wait
3	5	None	w	waiting	wait
3	6	None	w	waiting	wait
3	7	None	w	waiting	wait
3	8	None	w	waiting	wait
3	9	None	w	waiting	wait
3	10	None	w	waiting	wait
3	11	None	w	waiting	wait
3	12	None	w	waiting	wait
3	13	None	w	waiting	wait
3	14	None	w	waiting	wait
3	15	None	w	waiting	wait
3	16	None	w	waiting	wait
3	17	None	w	waiting	wait
3	18	None	w	ready to depart	move_forward
3	19	(9, 6)	w	moving	move_forward
3	20	(9, 5)	w	moving	move_forward
3	21	(9, 4)	w	moving	move_forward
3	22	(9, 3)	w	moving	move_forward
3	23	(8, 3)	n	moving	move_forward
3	24	(8, 2)	w	moving	move_forward
3	25	(8, 1)	w	moving	move_forward
3	26	(9, 1)	s	moving	move_forward
3	27	(10, 1)	s	moving	move_forward
3	28	(11, 1)	s	moving	move_forward
3	29	(12, 1)	s	moving	move_forward
3	30	(13, 1)	s	moving	move_forward
3	31	(13, 2)	e	moving	move_forward
3	32	(13, 3)	e	moving	move_forward
3	33	(13, 4)	e	moving	move_forward
3	34	(13, 5)	e	moving	move_forward
3	35	(13, 6)	e	moving	move_forward
3	36	(13, 7)	e	moving	move_forward
3	37	(13, 8)	e	moving	move_forward
3	38	(13, 9)	e	moving	move_forward
3	39	(13, 10)	e	moving	move_forward
3	40	(13, 11)	e	moving	move_forward
3	41	(13, 12)	e	moving	move_forward
3	42	(12, 12)	n	moving	move_right
3	43	(12, 13)	e	moving	move_forward
3	44	(12, 14)	e	moving	move_right
3	45	(13, 14)	s	moving	move_forward
3	46	(13, 15)	e	moving	move_forward
3	47	(13, 16)	e	moving	move_forward

B Tables for Energy Encoding

Table 5. Path taken by agent 0 in the energy encoding

agent	timestep	position	direction	status	given_command	energy
0	0	None	w	waiting	wait	0
0	1	None	w	waiting	wait	48
0	2	None	w	waiting	wait	0
0	3	None	w	waiting	wait	0
0	4	None	w	waiting	wait	0
0	5	None	w	waiting	wait	0
0	6	None	w	waiting	wait	0
0	7	None	w	waiting	wait	0
0	8	None	w	waiting	wait	0
0	9	None	w	waiting	wait	0
0	10	None	w	waiting	wait	0
0	11	None	w	waiting	wait	0
0	12	None	w	waiting	wait	0
0	13	None	w	waiting	wait	0
0	14	None	w	waiting	wait	0
0	15	None	w	waiting	wait	0
0	16	None	w	waiting	wait	0
0	17	None	w	waiting	wait	0
0	18	None	w	waiting	wait	0
0	19	None	w	waiting	wait	0
0	20	None	w	waiting	wait	0
0	21	None	w	waiting	wait	0
0	22	None	w	waiting	wait	0
0	23	None	w	waiting	wait	0
0	24	None	w	ready to depart	move_forward	0
0	25	(7, 6)	w	moving	move_forward	0
0	26	(7, 5)	w	moving	move_forward	2
0	27	(8, 5)	s	moving	move_forward	24
0	28	(8, 4)	w	moving	move_forward	24
0	29	(8, 3)	w	moving	move_forward	2
0	30	(8, 2)	w	moving	move_forward	2
0	31	(8, 1)	w	moving	move_forward	2
0	32	(9, 1)	s	moving	move_forward	24
0	33	(10, 1)	s	moving	move_forward	2
0	34	(11, 1)	s	moving	move_forward	2
0	35	(12, 1)	s	moving	move_forward	2
0	36	(13, 1)	s	moving	move_forward	2
0	37	(13, 2)	e	moving	move_forward	24
0	38	(13, 3)	e	moving	move_forward	2
0	39	(13, 4)	e	moving	move_forward	2
0	40	(13, 5)	e	moving	move_forward	2
0	41	(13, 6)	e	moving	move_forward	2
0	42	(13, 7)	e	moving	move_forward	2
0	43	(13, 8)	e	moving	move_forward	2
0	44	(13, 9)	e	moving	move_forward	2
0	45	(13, 10)	e	moving	move_forward	2
0	46	(13, 11)	e	moving	move_forward	2
0	47	(13, 12)	e	moving	move_forward	2
0	48	(12, 12)	n	moving	move_right	24
0	49	(12, 13)	e	moving	move_forward	24
0	50	(12, 14)	e	moving	move_right	2
0	51	(13, 14)	s	moving	move_forward	24
0	52	(13, 15)	e	moving	move_forward	24
0	53	(13, 16)	e	moving	move_forward	2

Table 6. Path taken by agent 1 in the energy encoding

agent	timestep	position	direction	status	given_command	energy
1	0	None	e	waiting	wait	0
1	1	None	e	waiting	wait	288
1	2	None	e	waiting	wait	0
1	3	None	e	waiting	wait	0
1	4	None	e	waiting	wait	0
1	5	None	e	waiting	wait	0
1	6	None	e	waiting	wait	0
1	7	None	e	waiting	wait	0
1	8	None	e	waiting	wait	0
1	9	None	e	waiting	wait	0
1	10	None	e	waiting	wait	0
1	11	None	e	waiting	wait	0
1	12	None	e	waiting	wait	0
1	13	None	e	waiting	wait	0
1	14	None	e	waiting	wait	0
1	15	None	e	waiting	wait	0
1	16	None	e	waiting	wait	0
1	17	None	e	waiting	wait	0
1	18	None	e	waiting	wait	0
1	19	None	e	waiting	wait	0
1	20	None	e	waiting	wait	0
1	21	None	e	ready to depart	move_forward	0
1	22	(12, 17)	e	moving	move_forward	0
1	23	(12, 18)	e	moving	move_forward	12
1	24	(12, 19)	e	moving	move_forward	12
1	25	(12, 20)	e	moving	move_forward	12
1	26	(12, 21)	e	moving	move_forward	12
1	27	(12, 22)	e	moving	move_forward	12
1	28	(11, 22)	n	moving	move_forward	144
1	29	(10, 22)	n	moving	move_forward	12
1	30	(9, 22)	n	moving	move_forward	12
1	31	(8, 22)	n	moving	move_forward	12
1	32	(7, 22)	n	moving	move_forward	12
1	33	(7, 21)	w	moving	move_forward	144
1	34	(7, 20)	w	moving	move_forward	12
1	35	(7, 19)	w	moving	move_forward	12
1	36	(7, 18)	w	moving	move_forward	12
1	37	(7, 17)	w	moving	move_forward	12
1	38	(7, 16)	w	moving	move_forward	12
1	39	(7, 15)	w	moving	move_forward	12
1	40	(7, 14)	w	moving	move_forward	12
1	41	(7, 13)	w	moving	move_forward	12
1	42	(7, 12)	w	moving	move_left	12
1	43	(8, 12)	s	moving	move_forward	144
1	44	(8, 11)	w	moving	move_forward	144
1	45	(8, 10)	w	moving	move_forward	12
1	46	(8, 9)	w	moving	move_forward	12
1	47	(8, 8)	w	moving	move_forward	12
1	48	(8, 7)	w	moving	move_forward	12

Table 7. Path taken by agent 2 in the energy encoding

agent	timestep	position	direction	status	given_command	energy
2	0	None	w	waiting	wait	0
2	1	None	w	waiting	wait	36
2	2	None	w	waiting	wait	0
2	3	None	w	ready to depart	move_forward	0
2	4	(12, 17)	w	moving	wait	0
2	5	(12, 17)	w	stopped	move_forward	36
2	9	(12, 16)	w	moving	move_forward	6
2	13	(12, 15)	w	moving	move_forward	6
2	17	(12, 14)	w	moving	move_forward	6
2	21	(12, 13)	w	moving	move_forward	6
2	25	(12, 12)	w	moving	move_forward	6
2	29	(12, 11)	w	moving	move_forward	6
2	33	(11, 11)	n	moving	move_forward	18
2	37	(10, 11)	n	moving	move_forward	6
2	41	(9, 11)	n	moving	move_forward	6
2	45	(8, 11)	n	moving	move_left	6
2	49	(8, 10)	w	moving	move_forward	18
2	53	(8, 9)	w	moving	move_forward	6
2	57	(8, 8)	w	moving	move_forward	6
2	61	(8, 7)	w	moving	move_forward	6

Table 8. Path taken by agent 3 in the energy encoding

agent	timestep	position	direction	status	given_command	energy
3	0	None	w	waiting	wait	0
3	1	None	w	waiting	wait	288
3	2	None	w	waiting	wait	0
3	3	None	w	waiting	wait	0
3	4	None	w	waiting	wait	0
3	5	None	w	waiting	wait	0
3	6	None	w	waiting	wait	0
3	7	None	w	waiting	wait	0
3	8	None	w	waiting	wait	0
3	9	None	w	waiting	wait	0
3	10	None	w	waiting	wait	0
3	11	None	w	waiting	wait	0
3	12	None	w	waiting	wait	0
3	13	None	w	waiting	wait	0
3	14	None	w	waiting	wait	0
3	15	None	w	waiting	wait	0
3	16	None	w	waiting	wait	0
3	17	None	w	waiting	wait	0
3	18	None	w	ready to depart	move_forward	0
3	19	(9, 6)	w	moving	move_forward	0
3	20	(9, 5)	w	moving	move_forward	12
3	21	(9, 4)	w	moving	move_forward	12
3	22	(9, 3)	w	moving	move_forward	12
3	23	(8, 3)	n	moving	move_forward	144
3	24	(8, 2)	w	moving	move_forward	144
3	25	(8, 1)	w	moving	move_forward	12
3	26	(9, 1)	s	moving	move_forward	144
3	27	(10, 1)	s	moving	move_forward	12
3	28	(11, 1)	s	moving	move_forward	12
3	29	(12, 1)	s	moving	move_forward	12
3	30	(13, 1)	s	moving	move_forward	12
3	31	(13, 2)	e	moving	move_forward	144
3	32	(13, 3)	e	moving	move_forward	12
3	33	(13, 4)	e	moving	move_forward	12
3	34	(13, 5)	e	moving	move_forward	12
3	35	(13, 6)	e	moving	move_forward	12
3	36	(13, 7)	e	moving	move_forward	12
3	37	(13, 8)	e	moving	move_forward	12
3	38	(13, 9)	e	moving	move_forward	12
3	39	(13, 10)	e	moving	move_forward	12
3	40	(13, 11)	e	moving	move_forward	12
3	41	(13, 12)	e	moving	move_forward	12
3	42	(12, 12)	n	moving	move_right	144
3	43	(12, 13)	e	moving	move_forward	144
3	44	(12, 14)	e	moving	move_right	12
3	45	(13, 14)	s	moving	move_forward	144
3	46	(13, 15)	e	moving	move_forward	144
3	47	(13, 16)	e	moving	move_forward	12